

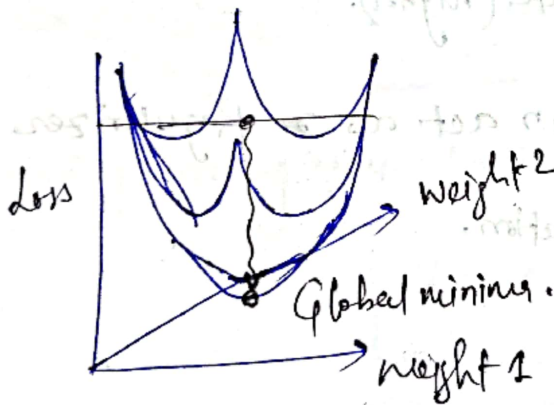
#>. Optimizers

Performance of NN how to speed the training

- weight initialization
- Batch Normalization.
- Activation function. it depend
- Optimizers.

#>. Optimizers

⇒ An optimizer is an algorithm that updates weights & bias of a neural network to minimize the loss (error).



⇒ You start with randomly & reach optimum solution or minimum.

⇒ Where loss is minimum you try to find value of weight which is corresponding to loss minimum.

⇒ Using optimizer Gradient descent

#>. Challenges :-

What challenges we face while using G.D :-

1. Calculating accurate value of learning rate.

2. Learning rate scheduling (you can automatically change your value of η according to pre defined schedule).

3. Multiple weights we need to find. (we can't keep separate weights G.D not provide this features).

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}$$

You are using this and run in loop with no. of epochs. then you get minimum.

4. local minimum (Best solⁿ comes in global minimum) but no more G.
 ↳ (little bit potential in stochastic G.D)
5. Saddle point (stop never change $\frac{\partial L}{\partial w} = 0$ when you try to update weights. $w_{new} = w_{old}$)

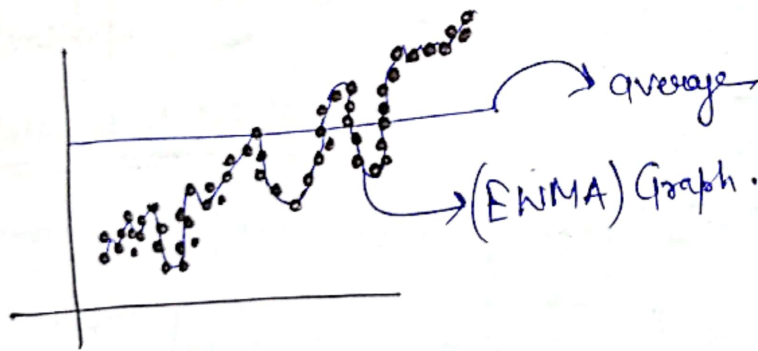
New optimizers :-

- ① Momentum
- ② Adagrad
- ③ NAG
- ④ RMS Prop
- ⑤ Adam (most use)

To understand this we learn or know about that topic Exponentially weighted moving Avg.

#> Exponentially weighted moving Avg (EWMA)

used in :-



- time series
- financial.
- Signal process
- Deep learning optimizers.

Formula of EWMA

$$V_t = \beta V_{t-1} + (1-\beta) Q_t$$

where β = constant value (0 to 1)

Some times $V_0 = 0$ & $V_0 = Q_0$ → more accurate.

$$V_1 = 0.9 \times V_0 + (1-0.9) \times 13$$

$$V_1 = 1.3$$

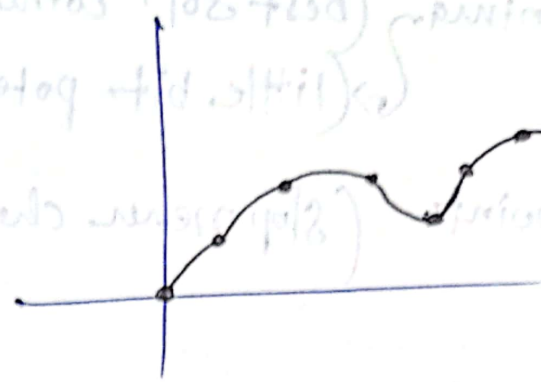
index	temp(°)
D ₁	25
D ₂	13
D ₃	17
D ₄	31
D ₅	43

$\beta = 0.9$ in (DL)

$$V_2 = 0.9 \times 1.3 + 0.1 \times 17$$

$$V = 2.87$$

in mean



$$\beta = \frac{1}{(1-\beta)}$$

if i took $\beta = 0.9$

$$\frac{1}{1-0.9} = \frac{1}{0.1} = 10$$

⇒ if you set value of $\beta = 0.9$ then your avg. at that point behave like last 10 days Avg.

if $\beta = 0.5$

$$\frac{1}{1-0.5} = \frac{1}{0.5} = 2$$



if you set $\beta = 0.5$ it behave like 2 days.



#>. Stochastic Gradient Descent with Momentum :-

SGD with momentum is an improved version of Stochastic Gradient descent (SGD). Instead of updating weight using only the current gradient it also considers the previous through a velocity term. This helps reduce oscillations in steep regions of the loss surface and accelerates convergence toward the minimum. The update rule is $V_t = \beta(V_{t-1}) + \eta \frac{\partial L}{\partial \theta}$ followed by $\cancel{W_{t+1} = W_t - V_t}$ $W = W - V_t$ where β is typically 0.9 as a result SGD with momentum converges faster and more smoothly than standard SGD.

⇒ It is very difficult to plot their graphs on 2-D. you can see in any animation tool or contour plots as well.

#>. NAG Optimizer (Nesterov Accelerated Gradient).

It is a way to implement momentum technique but improvement is this you damped your oscillations due to value of β which is decay constant.

Mathematical Intuition :-

momentum update rule :-

$$W_{t+1} = W_t - V_t$$

where :-

$$V_t = \beta V_{t-1} + \eta \nabla W_t$$

where $\beta \rightarrow$ decay factor.

where you reach in this graph to this point to this pt.

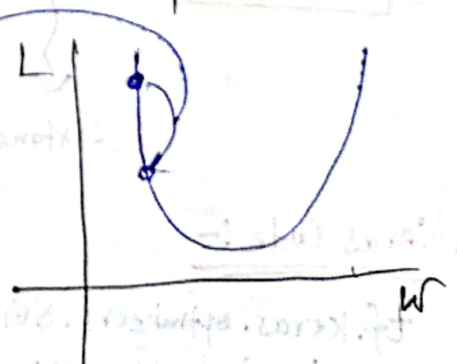
#>. this is intuition of momentum

this step decide in momentum

$$W_{t+1} = W_t - \left(\underbrace{\beta V_{t-1}}_{\text{Past velocity}} + \underbrace{\eta \nabla W_t}_{\text{Gradient at that point}} \right)$$

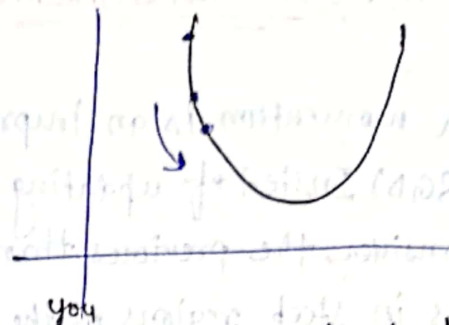
Past velocity.

Gradient at that point



#. Intuition of NAG

history of velocity + gradient at that point



⇒ In momentum at any point where you go you have to do two calculations

① history of velocity

② gradient at that point

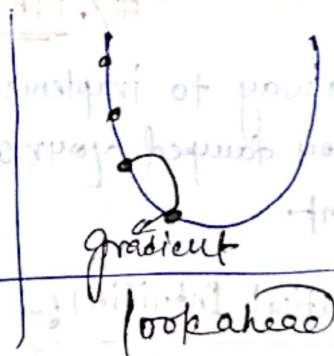
⇒ In NAG you take momentum jump and where you reach at that point you calculate gradient then according to this then you took another jump, that is difference b/w them and this is process where NAG is better performing than momentum.

l_a = look ahead.

$$W_{l_a} = W_t - \beta V_{t-1} \quad \text{momentum term update}$$

$$V_t = \beta V_{t-1} + \eta \nabla W_{l_a} \quad \text{slope}$$

$$W_{t+1} = W_t - V_t$$



$$V_t = (W_t - W_{l_a}) + \eta \nabla W_{l_a}$$

distance after momentum then calculating gradient

#. Keras Code:-

tf.keras.optimizers.SGD

learning_rate = 0.01, momentum = 0.0, nesterov = false, name = "SGD", **kwargs

SGD { momentum = 0.9 nesterov = false }

NAG { momentum = 0.9 nesterov = True }

##. Ada Grad Adaptive Gradient

you can't fix your learning rate, or you can adjust according to situation
you fix learning-rate.

- If your input features scale is different this situation you use Ada Grad
- If your features is (sparse) most of values are zero you can use Ada Grad.
- if your parameter update is big then you made your learning rate small.
if you multiple then update is small.
- Learning rate → small whose Gradient → big viceversa true

Formula:-

$$\begin{aligned} V_t &= V_{t-1} + (\nabla W_t)^2 \\ W_{t+1} &= W_t - \eta \frac{\nabla W_t}{\sqrt{V_t + \epsilon}} \end{aligned}$$

(V_t is Past Gradient Squared Sum.)
↳ ϵ is small no.

disadvantage :-

you can't reach your global minima. but theory is important to understand (RMS prop & Adam)

##. (RMS Prop) Root mean Square Prop

→ this optimization technique is an improvement over adagrad.

Adagrad :-

$$\begin{aligned} V_t &= V_{t-1} + (\nabla W_t)^2 \\ W_{t+1} &= W_t - \eta \frac{\nabla W_t}{\sqrt{V_t + \epsilon}} \end{aligned}$$

⇒ In V_t term you consider whole Gradient $(\nabla W_t)^2$ that why V_t term is bigger
⇒ for that V_t this term $\left(\frac{\eta \nabla W_t}{\sqrt{V_t + \epsilon}}\right)$ is smaller.

and update is negligible, and we stop to become (V_t) bigger this problem is solved by (RMS prop)

#1. RMS prop

$$V_t = \beta V_{t-1} + (1-\beta)(\nabla W_t)^2$$

$$\beta = 0.95$$

$$W_{t+1} = W_t - \frac{\eta \nabla W_t}{\sqrt{V_t + \epsilon}}$$

⇒ you took exponential avg you don't give more value of old Gradient epoch value you gave more value.

to reset Gradient epoch value that why our (V_t) term is not much bigger. and learning rate is not smaller and this term is not much smaller ~~$\frac{\eta \nabla W_t}{\sqrt{V_t + \epsilon}}$~~

⇒ Best optimizer RMS prop it compete (Adam. now)

$\left(\frac{\eta \nabla W_t}{\sqrt{V_t + \epsilon}} \right)$ and update are easily done.

#2. Adam

⇒ Adam → Adaptive moment Estimation.

⇒ most powerful optimizer

#3. you have to knowledge about all optimizers.

- ① Batch GD → It work better but Speed to reach Global minimum. the we use.
- ② Momentum → more oscillation and do to & for motion to reach Global minimum.
- ③ NAG → It damping (reduce) oscillation. and reach Global minimum. After that.
- ④ AdaGrad → Problem is Sparse in data. you go through Sparse & then non sparse to reach global minimum. but not much (fast).
→ it can't reach at global minimum. it decay learning rate but it done more that why it can't reach at global minimum.
- ⑤ RMS prop → It reach perfectly at Global minimum.

⇒ In optimizer we see:-

- ①. momentum
 - ②. learning-rate decay
- Adam merge both ideas.

⇒ Mathematical Formulation:-

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{V_t + \epsilon}} * M_t$$

Where:-

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla W_t \rightarrow \text{momentum}$$

$$V_t = \beta_2 V_{t-1} + (1 - \beta_2) (\nabla W_t)^2 \rightarrow \text{Adagrad}$$

⇒ After calculating this you have to do bias correction:-

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{V}_t = \frac{V_t}{1 - \beta_2^t}$$

★ In keras.

$$\begin{cases} \beta_1 = 0.9 \\ \beta_2 = 0.99 \end{cases} \left\{ \begin{array}{l} \eta = 0.1 \\ \eta = 0.01 \end{array} \right.$$

⇒ It show momentum behaviour in Animation.

⇒ Adam is good starting point in recent year.